

KANELÉ: Kolmogorov–Arnold Networks for Efficient LUT-based Evaluation

Duc Hoang*
dhoang@mit.edu
Massachusetts Institute of Technology
Cambridge, MA, USA

Aarush Gupta*
aarushg@mit.edu
Massachusetts Institute of Technology
Cambridge, MA, USA

Philip Harris
pcharris@mit.edu
Massachusetts Institute of Technology
Cambridge, MA, USA

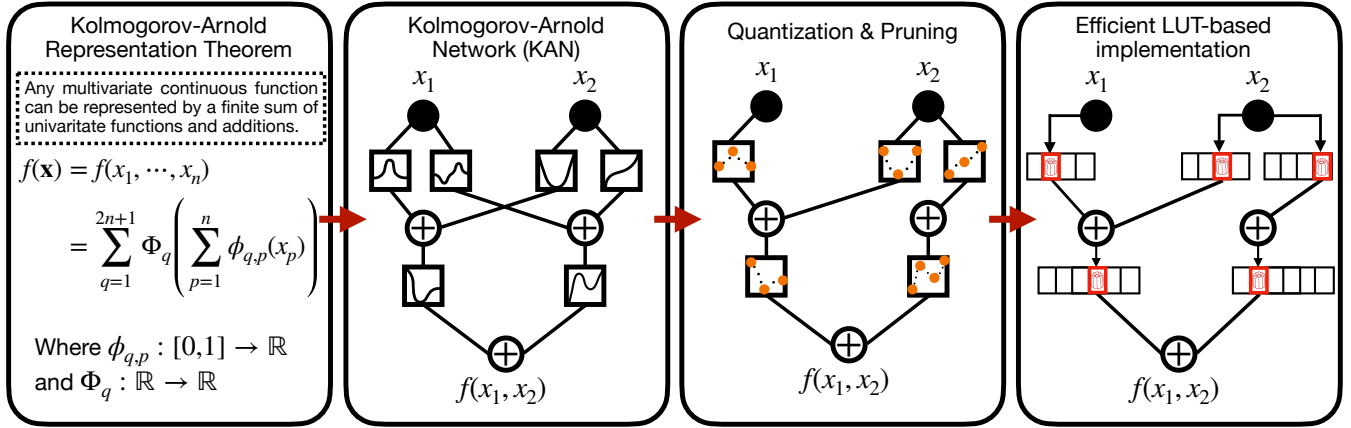


Figure 1: From the Kolmogorov-Arnold Representation Theorem to efficient KAN FPGA inference.

Abstract

Low-latency, resource-efficient neural network inference on FPGAs is essential for applications demanding real-time capability and low power. Lookup table (LUT)-based neural networks are a common solution, combining strong representational power with efficient FPGA implementation. In this work, we introduce KANELÉ, a framework that exploits the unique properties of Kolmogorov–Arnold Networks (KANs) for FPGA deployment. Unlike traditional multilayer perceptrons (MLPs), KANs employ learnable one-dimensional splines with fixed domains as edge activations, a structure naturally suited to discretization and efficient LUT mapping. We present the first systematic design flow for implementing KANs on FPGAs, co-optimizing training with quantization and pruning to enable compact, high-throughput, and low-latency KAN architectures. Our results demonstrate up to a 2700x speedup and orders of magnitude resource savings compared to prior KAN-on-FPGA approaches. Moreover, KANELÉ matches or surpasses other LUT-based architectures on widely used benchmarks, particularly for tasks involving symbolic or physical formulas, while balancing resource usage across FPGA hardware. Finally, we showcase the versatility of the framework by extending it to real-time, power-efficient control systems.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. *FPGA '26, Seaside, CA, USA*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2079-6/2026/02
<https://doi.org/10.1145/3748173.3779202>

CCS Concepts

- Computing methodologies → Machine learning algorithms;
- Hardware;

Keywords

Kolmogorov–Arnold Networks (KANs), FPGAs, Lookup tables (LUTs), Neural networks, Quantization, Pruning, Hardware–software codesign

ACM Reference Format:

Duc Hoang, Aarush Gupta, and Philip Harris. 2026. KANELÉ: Kolmogorov–Arnold Networks for Efficient LUT-based Evaluation. In *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '26)*, February 22–24, 2026, Seaside, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3748173.3779202>

1 Introduction

Lookup table (LUT) based neural networks have become a central paradigm for efficient FPGA inference, with designs such as NeuralLUT-Assemble [6], TreeLUT [22], DWN [7], and others [4, 45] demonstrating dramatic gains in area, latency, and power efficiency. These approaches highlight the advantages of rethinking neural networks around LUT primitives, though they remain largely confined to supervised learning and task-specific architectures.

In this work, we demonstrate that Kolmogorov–Arnold Networks (KANs) offer a principled foundation for LUT-based design. Inspired by the Kolmogorov–Arnold representation theorem, KANs replace the fixed activations of Multilayer Perceptrons (MLPs) with learnable edge functions and the matrix multiplication in MLPs

with summation at nodes (Fig. 1). This activation-centric formulation aligns naturally with LUTs: each learnable spline defined on a fixed domain can be quantized, pruned, and directly mapped to LUTs. In the literature, although KANs have been shown to outperform MLPs in settings such as PDE solving and scientific computing [26, 27], their practical deployment has been hindered by slow inference and costly hardware realizations [19, 39]. The only prior FPGA implementation concluded KANs were *impractical*, due to expensive spline evaluations and high resource usage [39].

This paper shows that by re-formulating KAN inference entirely in terms of LUTs, KANs are not only feasible but highly efficient in FPGA settings. Thus, our contributions are fourfold:

- (1) **FPGA-tailored KAN Architecture:** We present KANÉLÉ, named after the French pastry known for its compact form and rich structure [30]. At its core, the framework co-optimizes quantization, pruning, and mapping of KAN functions onto learned LUTs and additions, thereby minimizing memory and logic overhead. From a KAN research perspective, KANÉLÉ is the first FPGA-tailored formulation, eliminating BRAM/DSP usage, reducing latency by up to 2700×, and cutting resource usage by over 4000× compared to prior designs [39].
- (2) **High-Performance Realizations:** Unlike conventional LUT-based neural networks, where sequential LUT indexing makes pruning fundamentally incompatible with the model structure, KANÉLÉ leverages the additive independence of KANs to make pruning both natural and hardware efficient. Building on this architecture, KANÉLÉ delivers FPGA implementations that match or surpass other LUT-based neural designs, particularly for tasks well-suited to symbolic mapping. It sustains clock frequencies above 800 MHz across most benchmarks while achieving a state-of-the-art Area×Delay product and maintaining a balanced resource footprint.
- (3) **Open-source Framework:** We provide an automated software–hardware co-design flow that compiles KANs into optimized FPGA implementations within seconds, supporting reproducible studies across domains such as biology, physics, vision, signal processing, and tabular ML. Code is available at: <https://github.com/Duchstf/KANELE>
- (4) **Control Systems:** we extend KANÉLÉ beyond supervised learning to continuous control, showing on the HalfCheetah benchmark from OpenAI Gym [38] that a quantized KAN policy with $\sim 5\times$ fewer parameters than an MLP baseline policy achieves higher rewards, underscoring its suitability for resource-constrained, real-time control systems.

2 Background & Related Works

This section reviews Kolmogorov–Arnold Networks (KANs) and prior work on LUT-based neural network inference.

2.1 Kolmogorov–Arnold Networks

Kolmogorov–Arnold Networks (KANs) replace the fixed activation functions and matrix multiplications of MLPs with learnable spline-based functions on network edges [27]. This activation-centric formulation improves expressiveness and interpretability, often achieving comparable accuracy with fewer parameters and operations.

Since their introduction, KANs have inspired extensive follow-up work, including theoretical analyses [43, 46], architectural extensions (e.g., convolutional [9, 14], temporal [17], and Fourier-based variants [20, 47]), and applications across scientific modeling and data-driven tasks [12, 18, 24, 44].

Despite this rapid progress, recent surveys identify computational efficiency and hardware implementation as key open challenges [19, 21, 35]. To date, efficient hardware realization remains largely unexplored, with one early attempt concluding that a direct FPGA implementation incurs prohibitive resource and latency costs compared to MLPs [39]. Our work directly challenges this conclusion by demonstrating that the activation-centric design of KANs is, in fact, exceptionally well-suited for hardware acceleration through a LUT-based paradigm.

2.2 LUT-based Neural Networks

LUT-based neural networks aim to replace arithmetic-heavy MAC operations with precomputed function evaluations stored in LUTs, exploiting the abundant and low-latency logic resources of FPGAs. Pioneering frameworks like LUTNet [42] and LogicNets [40] first demonstrated the replacement of arithmetic with direct LUT mappings. Subsequent works generalized this concept to approximate more complex functions [4, 5] and improved scalability using additive or modular ensembles [6, 29, 45]. This design philosophy has also been used to efficiently implement other machine learning models, such as gradient-boosted decision trees [22]. Another related approach is the family of Weightless Neural Networks (WNNs), which stores learned patterns directly in LUTs [3, 7, 31, 36, 37], though often at the cost of representational power.

Conceptually, KANs are close to PolyLUT [4], PolyLUT-Add [29], and DWNs [7] but possess distinct structural properties. While PolyLUT tabulates multivariate polynomials, which theoretically allows the native representation of arbitrary products $p(\mathbf{x}) = \prod_i x_i$, this approach suffers from exponential LUT growth relative to input dimension. In contrast, KANs decompose functions into sums of tabulated univariate splines. Although this formulation relies on layer composition to approximate the multiplicative terms inherent to PolyLUT, it yields linear scaling with input dimension and an additive structure that is naturally amenable to pruning. While this formulation doesn’t explicitly represent pure multiplicative terms, in practice, compositions of low-order (≤ 3) splines approximate such interactions effectively. Moreover, DWN’s full binarization of inputs and LUTs hinders generalization beyond classification, while KANÉLÉ supports higher-precision arithmetic for tasks such as autoencoding and continuous control. Finally, DWN’s finite-difference differentiability may further constrain optimization flexibility compared to KANÉLÉ gradient descent.

3 KAN Architecture and Quantization-Aware Training and Pruning

We design the KANÉLÉ framework for KAN FPGA deployment using quantization-aware training and pruning, enabling efficient hardware translation while preserving consistency between training and inference.

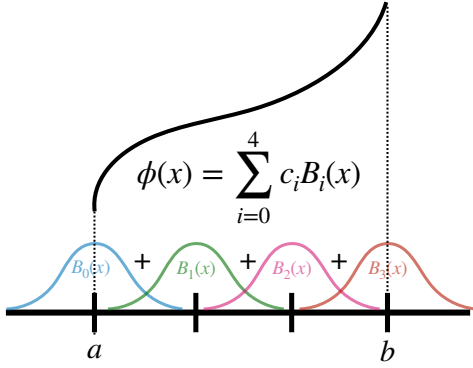


Figure 2: A KAN activation $\phi(x)$ represented as a linear combination of B-spline basis functions $B_i(x)$ on a grid over $[a, b]$: $\phi(x) = \sum_i c_i B_i(x)$. Trainable coefficients c_i control the overall function shape.

3.1 KAN Architecture with Learnable Activation Functions

Before introducing quantization and pruning, we first outline the core architecture. Unlike MLPs, KANs replace fixed nonlinearities with *learnable activation functions*, each modeled as a linear combination of B-spline basis functions with trainable coefficients. B-splines are piecewise polynomials defined on a grid, providing smoothness, locality, and efficient nonlinear parameterization. More generally, activations can be expressed in other orthogonal bases, such as Fourier series [20, 25, 47].

A KAN layer with d_{in} inputs and d_{out} outputs is represented as a matrix of 1D learnable functions

$$\Phi = \{\phi_{q,p}\}, \quad p = 1, \dots, d_{\text{in}}, \quad q = 1, \dots, d_{\text{out}}, \quad (1)$$

where each $\phi_{q,p}$ is trainable (Fig. 2).

For improved convergence, each $\phi_{q,p}$ combines a base activation $\phi(\cdot)$ with B-splines $\{B_{p,k}(\cdot)\}$:

$$\phi_{q,p}(x_p) = w_{q,p}^{\text{base}} \phi(x_p) + \sum_{k=1}^{G+S} w_{q,p,k}^{\text{spline}} B_{p,k}(x_p), \quad (2)$$

where $w_{q,p}^{\text{base}}$ and $w_{q,p,k}^{\text{spline}}$ are trainable. Splines are defined on a grid of size G and order S within a fixed domain $[a, b]$.

Given $x_l \in \mathbb{R}^{d_{\text{in}}}$, the output is

$$(x_{l+1})_j = \sum_{i=1}^{d_l} \phi_{j,i}(x_{l,i}), \quad j = 1, \dots, d_{l+1}, \quad (3)$$

or in compact form

$$x_{l+1} = \Phi_l(x_l), \quad (4)$$

with Φ_l the function matrix of layer l . A L -layer KAN is thus

$$\text{KAN}(x) = \Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_0(x). \quad (5)$$

As illustrated in Fig. 1, KANs extend MLPs by learning activations directly, offering greater representational flexibility while preserving a structured, layer-wise graph.

3.2 Quantization-Aware Training

For efficient FPGA deployment, we adopt quantization-aware training (QAT) via AMD’s Brevitas library [16]. Quantizers are placed at the network input and after each KAN layer, ensuring that training adapts to the required hardware precision.

For a layer l with output $x_{l+1} \in \mathbb{R}^{d_{l+1}}$, the quantized output is

$$x_{l+1,q} = q_l(x_{l+1}), \quad (6)$$

where $q_l(\cdot)$ is the layer quantizer. Similarly, an input quantizer $q_l(\cdot)$ is applied to x_0 .

The layer output quantizer performs n_l -bit uniform quantization:

$$x_{l+1,q} = s_l \cdot \text{Quantize}[n_l] \left(\frac{\text{clip}(x_{l+1}, a, b)}{s_l} \right), \quad (7)$$

where s_l is a learnable scale (fixed at inference), and $[a, b]$ is the shared quantization domain (Fig. 3).

The input quantizer incorporates both scale s_l and bias b_l to handle asymmetric distributions:

$$x_{0,q} = s_l \cdot \text{Quantize}[n_l] \left(\frac{\text{clip}(x_0, a, b)}{s_l} + b_l \right). \quad (8)$$

During RTL generation, s_l and b_l are fixed for deterministic behavior.

In practice, input preprocessing is realized by a batch normalization (zero mean, unit variance) followed by a `ScalarBiasScale` block introducing b_l and s_l . At inference, BN statistics are folded into these constants, yielding an affine shift–scale, clipping, and quantization. This design preserves LUT-based compatibility while avoiding the overhead of full batch normalization.

During training, quantizer gradients are approximated using the straight-through estimator (STE):

$$\frac{\partial q(x)}{\partial x} \approx 1, \quad (9)$$

which allows gradient flow through quantized operations without modification.

3.3 Pruning via Norm-Based Selection

To reduce resource usage, we prune spline connections by evaluating their contribution over the input domain. In contrast to

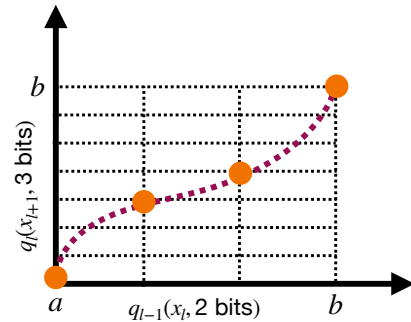


Figure 3: Layer-wise uniform quantization. Here, 2-bit inputs $q_{l-1}(x_l)$ are mapped to 3-bit outputs $q_l(x_{l+1})$ over the fixed range $[a, b]$. Orange markers indicate quantization levels; the dotted curve is the underlying continuous mapping.

conventional LUT-based neural networks—which rely on sequential LUT indexing, making every LUT entangled with the next and thus nearly impossible to prune without breaking the model—KANELÉ exploits the inherently additive structure of KANs, where each LUT contributes independently to a summation. This independence makes pruning both mathematically natural and directly compatible with FPGA hardware. The original KAN paper emphasizes efficient pruning as a key advantage of KAN’s edge-centric architecture, and we extend this insight to demonstrate a distinct advantage over node-based LUT networks for efficient hardware translation [27].

For each pair (i, j) of input and output neurons, we compute the activation of the spline component:

$$f_{p \rightarrow q}(x) = \sum_{k=1}^{G+S} w_{q,p,k}^{\text{spline}} B_{p,k}(x). \quad (10)$$

Its importance is measured via the ℓ_2 norm across a sampled input grid $\mathcal{X}$ consistent with its quantization level:

$$\|f_{p \rightarrow q}\|_2 = \left(\sum_{x \in \mathcal{X}} |f_{p \rightarrow q}(x)|^2 \right)^{1/2}. \quad (11)$$

A structured pruning mask is then applied:

$$m_{q,p} = \begin{cases} 1, & \|f_{p \rightarrow q}\|_2 > \tau(t), \\ 0, & \text{otherwise,} \end{cases} \quad (12)$$

where $\tau(t)$ is a pruning threshold that changes as a function of epochs (t) with

$$\tau(t) = T \exp \left(-\ln 20 \cdot \frac{\max(t, t_0)}{t_f - t_0} \right).$$

This pruning threshold corresponds to an exponential warmup, where pruning starts on epoch t_0 and increases exponentially, hitting 95% of the full pruning threshold T on target epoch t_f . This allows us to control pruning dynamics to avoid interference with proper training. Backward pruning is additionally applied if the corresponding output neuron has no active connections in the subsequent layer, ensuring consistent sparsity propagation.

3.4 KAN Hyperparameters

To summarize, the training and deployment of KANs involves hyperparameters which can be broken up into three main classes: spline representation hyperparameters, hardware architecture hyperparameters, and pruning hyperparameters. The descriptions and impact of each hyperparameter are detailed in Table 1. The joint optimization of these parameters provides a flexible design space that balances learning capacity with hardware efficiency in FPGA deployments.

4 LUT-Based KAN Architecture

This section outlines our end-to-end mapping of trained KANs to synthesizable VHDL RTL and associated pipelining strategies. Our end-to-end toolflow currently supports the basic KAN architecture using B-splines, as in the original paper [27]. Extensions to other bases or architectures such as convolutions or transformers are

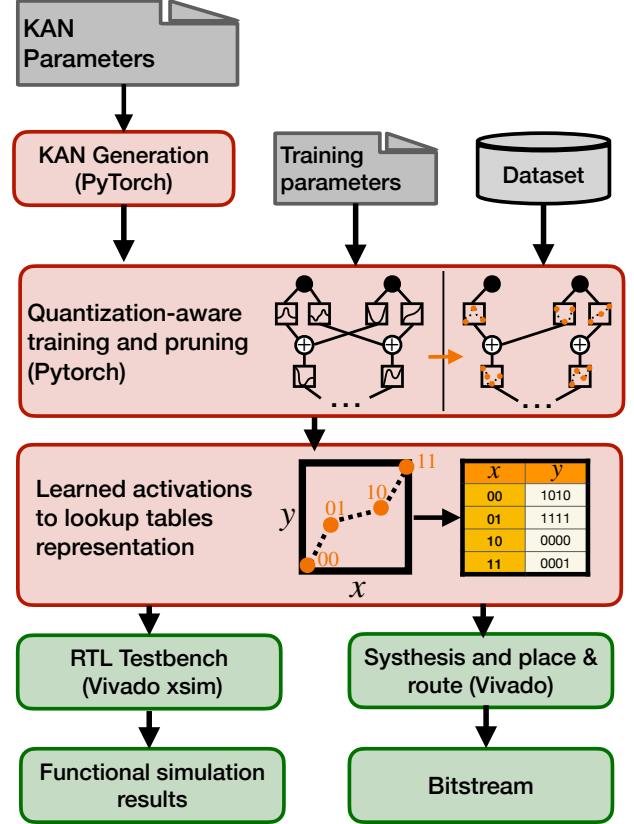


Figure 4: Visualization of the KAN to FPGA implementation toolflow.

feasible. The framework is designed for usability—anyone familiar with training MLPs can readily train and deploy KANELÉ.

4.1 Toolflow

A high-level overview of the toolflow stages is shown in Figure 4. This push-button workflow removes manual RTL work: starting from a trained PyTorch checkpoint, it deterministically emits RTL, memory images, and build/simulation scripts, enabling rapid deployment of arbitrary KAN topologies to FPGA and bitstream generation via Vivado.

4.1.1 Training. The training process begins by specifying the KAN hyperparameters (see Section 3.4) together with the dataset to be used. The user may freely select the optimizer and learning-rate schedule best suited to their task. In our implementation, we adopt the PyTorch AdamW optimizer [28] as the default choice due to its robustness in handling weight decay. The model is then trained with the QAT and pruning mechanism described in Section 3.2 and Section 3.3, respectively, to reduce resource usage and latency, while preserving accuracy. This step produces compact learned representations for KAN that can be efficiently mapped to LUT representations for FPGA deployment.

Table 1: Summary of KAN training parameters. The first group influences the accuracy through spline representation, the second group encodes the hardware architecture, and the third group determines pruning policy which affects hardware architecture.

Symbol	Description	Impact
G	Grid size (number of intervals)	Controls spline resolution; accuracy only
$[a, b]$	Grid range (spline domain)	Defines support of basis functions; accuracy only
S	Spline order	Smoothness and flexibility; accuracy only
d_l	Layer dimensions	Affects model capacity and resource usage
n_l	Layer bitwidth (QAT precision)	Direct trade-off between accuracy and resource cost
T	Pruning threshold	Governs sparsity and LUT reduction and accuracy tradeoff
t_0	Warmup start epoch	Determines when pruning warmup starts
t_f	Warmup target epoch	Affects pruning warmup rate

4.1.2 KAN to Logical-LUTs Conversion. Following [6], we denote lookup tables extracted from the network as *Logical-LUTs* (L-LUTs), and FPGA fabric resources as *Physical-LUTs* (P-LUTs). From a trained, pruned, and quantized PyTorch KAN, each surviving connection is translated into an L-LUT. For every active edge, the input state space is enumerated and the KAN layer’s pre-activation response is evaluated and quantized. This produces per-connection truth tables stored as compact JSON files, yielding a deterministic, bit-accurate mapping of the model into integer-valued L-LUTs. The representation preserves quantization and sparsity, enabling efficient FPGA deployment.

4.1.3 RTL File Generation. From the L-LUT graph, we generate a complete RTL design for FPGA deployment. The tool emits VHDL sources for the KAN core, per-layer packages, LUT entities, and memory initialization files encoding the truth tables. A configuration package specifies bit widths, signal types, and accumulator sizes. Each L-LUT is instantiated as a memory-mapped component, organized into layers, with balanced adder trees for output accumulation. Pipeline registers are inserted between layers to improve clocking and shorten critical paths. The result is a self-contained firmware bundle that includes simulation testbenches, initialization vectors, and Vivado build scripts—supporting functional simulation, latency evaluation, and FPGA synthesis.

4.1.4 Synthesis and Place & Route. We synthesize the generated RTL with Vivado 2024.1, targeting the xcvu9p-flgb2104-2-i FPGA for benchmarking against LUT-based networks, and the xczu7ev-ffvc1156-2-e FPGA for comparison with prior KAN works. To ensure fairness and consistency with prior works, we use settings that isolate core delay and area, specifically Vivado’s Flow_PerfOptimized_high mode with Out-of-Context synthesis, allowing each module to be compiled independently. While the maximum clock is ultimately limited by the FPGA’s global clock, LUT-centric designs such as KANELÉ typically sustain high frequencies, making the computational core unlikely to be the critical path in larger systems. The target clock period is therefore chosen relative to network size, following prior work.

4.2 Pipelining Strategies

Efficient pipelining is crucial for achieving high FPGA clock frequencies while maintaining low latency across KAN layers. We introduce pipelining at two levels: (i) within adder trees that accumulate outputs of multiple Logical-LUTs (L-LUTs) per channel, and (ii) between consecutive network layers.

Adder Tree Pipelining. Each neuron computes a weighted sum of active inputs via a reduction tree over L-LUT outputs. A naïve single-stage sum creates a long combinational path, thereby limiting frequency. Instead, we implement a *balanced, pipelined adder tree* with registers after each stage. At each stage up to n_{add} inputs are combined, reducing fan-in and distributing additions over multiple cycles. The depth is

$$\text{depth}_\ell = \left\lceil \log_{n_{\text{add}}} (N_\ell) \right\rceil,$$

as shown in Figure 5 for $n_{\text{add}} = 2$. At the end of the adder tree, quantization and saturation of the sum are performed to make the output consistent with the subsequent layer’s input. This is taken into account during training, preventing any degradation in accuracy.

Inter-Layer Pipelining. Pipeline registers are also inserted between layers, capturing saturated outputs before feed-forward. This isolates LUT evaluation, summation, and activation in time, minimizing critical paths and balancing latency. Register insertion is automated in RTL generation, ensuring deep pipelining for arbitrary KAN topologies.

Limitations. As a LUT-based model, KAN inherits known limitations. LUT size scales exponentially with input bitwidth, though only linearly with fan-in [4, 5]. For high-dimensional inputs, preserving KAN’s structure requires long adder chains, reducing throughput and increasing resource usage. Adder trees sustain high clock frequencies by adding pipeline stages, at the cost of extra clock cycles. Thus, image tasks like MNIST require aggressive pruning to remain resource-feasible, with some accuracy loss.

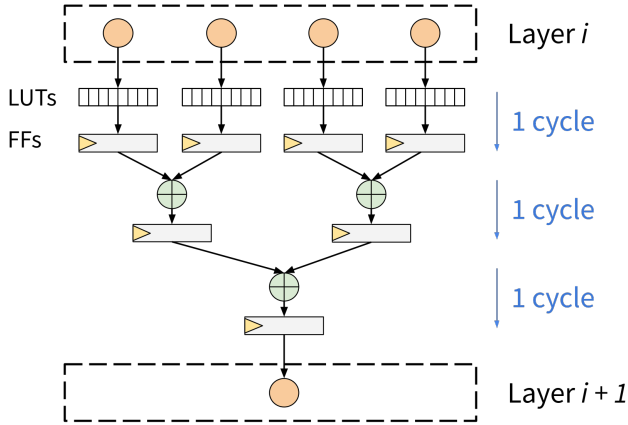


Figure 5: Balanced, pipelined adder tree for computing one neuron activation with $n_{add} = 2$.

5 Experimental Results

5.1 Benchmarks

We evaluate KAN across three domains of datasets: (i) widely adopted benchmark datasets from the LUT-based neural network literature, (ii) synthetic and tabular datasets previously used in KAN FPGA benchmarking [39], and (iii) MLPerf Tiny datasets, which provide real-world tasks with more complex modalities and objectives. Together, these domains form a diverse testbed to study KAN across different tasks and levels of complexity. Below we briefly describe each dataset.

5.1.1 LUT-based Neural Network Benchmarks.

- **MNIST:** A large-scale handwritten digit recognition dataset containing 60,000 training and 10,000 test images of size 28×28 , labeled across 10 classes (digits 0–9) [13].
- **JSC OpenML:** A tabular dataset [11] from the JSC suite, consisting of 16 jet substructure input features and a 5-class jet classification task. It has been widely used in comparisons of LUT-based networks. This version contains 830,000 instances and is known to exhibit easier convergence, possibly due to improved data curation [6].
- **JSC CERNBox:** Another dataset [33] from the JSC benchmark, involving the same jet tagging task. It comprises 986,806 instances and is generally considered more challenging, as models trained on it tend to achieve lower accuracies due to the increased dataset complexity.

5.1.2 KAN FPGA Benchmarks.

- **Moons:** A synthetic two-class dataset [34] commonly used for testing nonlinear decision boundaries. Each point lies in one of two interleaving half-moon shapes with added Gaussian noise.
- **Wine:** A dataset from the University of California, Irvine (UCI) Machine Learning Repository [2], containing 13 physicochemical attributes of wine samples classified into 3 quality categories.

- **Dry Bean:** Another UCI dataset [1] with 16 numerical features representing bean shape and texture, used for classifying 7 different bean varieties.

5.1.3 MLPerf Tiny Benchmarks.

- **ToyADMOS:** An audio anomaly detection dataset featuring sound files from both normally functioning and defective toy cars. An autoencoder is trained on sliding windows of the downsampled mel spectrogram (input size 64). For classification, the mean reconstruction loss across all sliding window spectrograms of an audio file is computed, and a fixed threshold is applied to label the sample as an anomaly. Notably, this benchmark presents significantly more complex inputs compared to the LUT-NN and KAN-FPGA benchmarks while also utilizing a non-classification objective (reconstruction loss). [8].

These datasets span a spectrum of complexity, from low-dimensional toy datasets (Moons) to high-dimensional real-world data (JSC, MNIST, ToyADMOS), ensuring that both the representational capacity and hardware efficiency of KANÉLÉ are thoroughly evaluated.

5.2 Training Parameters

KAN hyperparameters are straightforward to manage. The spline-related parameters (G , $[a, b]$, S) only affect accuracy and can often be set to robust defaults since they do not impact hardware resources.

In practice, balancing model performance with hardware efficiency centers on tuning three key parameters: the layer dimensions (d_l), bitwidth (n_l), and pruning threshold (T). These directly control the model’s capacity, numerical precision, and sparsity. As demonstrated in Table 2, this allows the quantized and pruned KANs to achieve competitive accuracy—even outperforming floating-point versions on datasets like Wine (98.2%)—while being optimized for an efficient FPGA implementation.

5.3 Comparison with LUT-NN Architectures

We benchmarked KANÉLÉ against state-of-the-art LUT-based architectures on three datasets: JSC CERNBox, JSC OpenML, and MNIST (Table 3). It should be noted that, as detailed in Table 2, we assume the same input bitwidth compared to prior works, which is not the case for DWN [7] that uses a thermometer encoding to assign distinct floating-point thresholds to each feature, leading to potentially large overhead.

JSC CERNBox. On the more difficult JSC CERNBox dataset, KANÉLÉ achieves the highest accuracy (75.1%), tying with Neural-LUT while also using $18\times$ less LUTs and two orders of magnitude fewer resources. Compared to the best prior model when considering resources, NeuralLUT-Assemble, we obtain slightly higher accuracy with $1.7\times$ fewer LUTs, over $2.4\times$ higher F_{max} , and the lowest Area $\times$ Delay product (4.1×10^4). In contrast, alternatives such as AmigoLUT, PolyLUT, and LogicNets consume an order of magnitude more LUTs, suffer lower accuracy, or are limited to F_{max} in the 200–500 MHz range. Overall, KANÉLÉ sits on the Pareto frontier of accuracy versus efficiency, establishing it as the most efficient solution for this task.

Table 2: Accuracy comparison of MLP Floating Point and KAN Floating Point and Quantized models on benchmark datasets. All models have the same dimensions listed. The floating point versions only use the layer size (d_l) parameter.

Datasets	G	$[a, b]$	S	d_l	n_l	T	Accuracy (%) / AUC		
							MLP FP	KAN FP	KAN Quantized & Pruned
KAN FPGA Benchmarks [39]									
Moons	6	[-8, 8]	3	[2, 2, 1]	[6, 5, 8]	0.	87.2	97.7	97.4
Wine	6	[-8, 8]	3	[13, 4, 3]	[6, 7, 8]	0.	96.3	98.1	98.2
Dry Bean	6	[-8, 8]	3	[16, 2, 7]	[6, 6, 8]	0.	90.9	92.2	92.1
LUT-based neural network benchmarks									
MNIST	30	[-8,8]	3	[784, 62, 10]	[1, 6, 6]	1.	96.7	97.9	96.3
JSC CERNBox	30	[-2, 2]	10	[16, 12, 5]	[8, 8, 6]	0.14	73.0	75.1	75.1
JSC OpenML	40	[-2, 2]	10	[16, 8, 5]	[6, 7, 6]	0.9	76.5	76.5	76.0
MLPerf Tiny Benchmark									
ToyADMOS	30	[-2, 2]	10	[64, 16, 8, 16, 64]	[7, 8, 8, 7, 8]	0.9	0.80	0.83	0.83

JSC OpenML. On the easier JSC OpenML dataset, most neural networks plateau around 76% accuracy. KAN-LUT reaches this level (76.0%) while using only 1232 LUTs—the fewest among all models and up to 51× fewer than the hls4ml implementation. Compared to NeuralLUT-Assemble, KAN-LUT requires 1.44× fewer LUTs and achieves a slightly higher $F_{\max}$ (987 vs. 941 MHz), though its longer latency (7.1 ns vs. 2.1 ns) results in a larger Area×Delay. Other networks achieve marginally higher accuracy, but only at the cost of substantially greater resource usage and latency. Overall, KAN-LUT offers one of the best trade-offs between accuracy and efficiency on this task.

MNIST. On the MNIST dataset, KANELÉ achieves a high accuracy of 96.3%, though some specialized models like NeuralLUT-Assemble and DWN reach close to 98%. In terms of hardware resources, DWN is the most compact with 2092 LUTs. KANELÉ, with 3809 LUTs, is still significantly more efficient than the majority of other high-accuracy models. For instance, it uses over 20 times fewer LUTs than PolyLUT (75131 LUTs) while achieving only 2% lower accuracy. Architectures like NeuralLUT-Assemble and TreeLUT excel in latency (2.1 ns and 2.5 ns) and achieve the best Area×Delay products. This suggests that the architectural priors of these specialized models may be better aligned with the spatial structure inherent in image data than the function-approximation paradigm of KANs. Consequently, extending KANELÉ toward convolutional architectures appears to be a promising direction for future work on image-based tasks.

In summary, KANELÉ consistently demonstrates an exceptional trade-off between predictive accuracy and hardware resource utilization across different benchmarks. KAN achieves an efficient balance of FPGA resources by leveraging both LUTs and FFs in a complementary manner. It particularly excels in complex, resource-intensive tasks like the JSC CERNBox benchmark, where it sets a new state-of-the-art in terms of the Area×Delay product. Based on the nature of these datasets, it can also be inferred that KANELÉ is better suited for tasks involving symbolic or physical formulas between the input and outputs (e.g., JSC variants), which naturally aligns with the Kolmogorov–Arnold representation theorem.

5.4 Comparison with Prior KAN-FPGA Literature

We benchmarked KANELÉ against the KAN FPGA implementation by Tran et al. [39] on the Moons, Wine, and Dry Bean datasets. The results, detailed in Table 4, demonstrate that KANELÉ offers a dramatic improvement in hardware efficiency and performance while maintaining or exceeding the accuracy of the previous work.

Through its LUT-based approach, our implementation completely eliminates the need for BRAM and DSP blocks, which are heavily utilized in the implementation by Tran et al. This leads to a massive reduction in the overall hardware footprint. For instance, on the Dry Bean dataset, KANELÉ uses only 402 LUTs and 471 FFs, whereas the previous work consumes over 1.6 million LUTs and 734,000 FFs—a reduction of more than 4000× in LUTs.

KANELÉ also achieves high maximum frequencies (up to 1736 MHz) and very low latencies. On the Dry Bean benchmark, for example, our model’s latency is 7.1 ns, a speedup of over 2600× compared to the 18,960 ns reported by Tran et al. Consequently, KANELÉ achieves an excellent Area×Delay product across all benchmarks.

5.5 Comparison with hls4ml MLPerf Tiny

To understand the performance of KANELÉ on more complex datasets, we compare the framework to hls4ml [10] on the ToyADMOS dataset, part of the MLPerf Tiny benchmark suite. Other LUT-based neural networks have not attempted this benchmark, possibly due to its high complexity and/or non-classification-based training scheme. The results in Table 5 indicate that KANELÉ achieves substantially better performance than prior approaches in terms of both resource efficiency, latency, and power. These improvements highlight the potential of KANELÉ for future studies involving more complex datasets as well as tasks beyond classification. Specifically, KANELÉ eliminates the need for BRAM, LUTRAM, and DSPs, while reducing LUT usage by 41.7% and FF usage by 71.4% relative to hls4ml. In terms of performance, KANELÉ delivers 330× higher throughput, 643× lower latency and a 9,840× reduction in energy per inference. These results establish KANELÉ as a highly efficient alternative to existing FPGA neural implementations, with strong

Table 3: Evaluation of KANELÉ against state-of-the-art ultra-low-latency, resource-efficient LUT-based network architectures. Results are reported after performing *out-of-context* synthesis and place-and-route. Input bit-widths are consistent with those used in prior works for fair comparison.

Dataset	Model	Accuracy (%)	LUT	FF	DSP	BRAM	$F_{\max}$ (MHz)	Latency (ns)	Area×Delay (LUT×ns)
JSC CERNBox	KANELÉ	75.1	5034	1917	0	0	870	8.1	4.1×10^4
	NeuraLUT-Assemble [6]	75.0	8539	1332	0	0	352	5.7	4.87×10^4
	AmigoLUT-NeuraLUT [45]	74.4	42742	4717	0	0	520	9.6	4.10×10^5
	PolyLUT-Add [29]	75.0	36484	1209	0	0	315	16	5.84×10^5
	NeuraLUT [5]	75.1	92357	4885	0	0	368	14	1.29×10^6
	PolyLUT [4]	75.0	246071	12384	0	0	203	25	6.15×10^6
	LogicNets [40]	72.0	37931	810	0	0	427	13	4.93×10^5
JSC OpenML	KANELÉ	76.0	1232	900	0	0	987	7.1	8.7×10^3
	NeuraLUT-Assemble [6]	76.0	1780	540	0	0	941	2.1	3.92×10^3
	TreeLUT [22]	75.6	2234	347	0	0	735	2.7	6.03×10^3
	DWN [7]	76.3	4972	3305	0	0	827	7.3	9.07×10^4
	hls4ml (Fahim et al.) [15]	76.2	63251	4394	38	0	200	45	2.85×10^6
MNIST	KANELÉ	96.3	3809	4133	0	0	864	9.3	3.5×10^4
	NeuraLUT-Assemble [6]	97.9	5070	725	0	0	863	2.1	1.06×10^4
	TreeLUT [22]	96.6	4478	597	0	0	791	2.5	1.12×10^4
	DWN [7]	97.8	2092	1757	0	0	873	9.2	1.92×10^4
	PolyLUT-Add [29]	96.0	14810	2609	0	0	625	10	1.48×10^5
	AmigoLUT-NeuraLUT [45]	95.5	16081	13292	0	0	925	7.6	1.22×10^5
	NeuraLUT [5]	96.0	54798	3757	0	0	431	12	6.58×10^5
	PolyLUT [4]	97.5	75131	4668	0	0	353	17	1.38×10^6
	FINN [41]	96.0	91131	—	0	5	200	310	2.82×10^7
	hls4ml (Ngadiuba et al.) [32]	95.0	260092	165513	0	345	200	190	4.94×10^7

Table 4: FPGA resource utilization and latency of KAN models on Moons, Wine, and Dry Bean benchmarks as used in [39]

Dataset	Model	Accuracy (%)	$F_{\max}$ (MHz)	BRAM	DSP	FF	LUT	Latency (cycles)	Latency (ns)	Area×Delay (LUT × ns)
Moons	KANELÉ	97	1736	0	0	57	67	5	2.9	1.7×10^2
	KAN (Tran et al) [39]	97	-	10	120	8622	17877	128	1280	-
Wine	KANELÉ	98	983	0	0	686	534	6	6.1	8.8×10^3
	KAN (Tran et al) [39]	97	-	132	950	74741	146843	688	6880	-
Dry Bean	KANELÉ	92	842	0	0	471	402	6	7.1	3.3×10^3
	KAN (Tran et al) [39]	92	-	781	9111	734544	1677558	1896	18960	-

Table 5: Comparison of FPGA resource utilization, latency, and power consumption for the anomaly detection task on the ToyADMOS time series dataset in the MLPerf Tiny Benchmark, evaluated on the xc7a100t-1csg324 FPGA [8].

Dataset	Model	AUC	BRAM (36kb)	DSP	FF	LUT	LUTRAM	II (clocks)	Throughput (inf/s)	Latency (μ s)	Energy/inf. (μ J)
ToyADMOS	KANELÉ	0.83	0	0	17,643	29,981	0	1	228 M	0.07	0.01
	hls4ml (MLPerf Tiny v0.7) [10]	0.83	22.5	207	61,639	51,429	5,780	144	694 k	45	98.4

potential for scaling to more complex datasets and tasks beyond classification.

5.6 Ablation Study

We perform an ablation study for KANELÉ using the JSC OpenML dataset, shown in Figure 6. This analysis isolates the effect of four key design factors—accuracy, pruning, hidden layer width, and model bitwidth—on FPGA resource utilization (LUTs and FFs). It is seen that pruning and quantization provide the most effective levers for controlling hardware footprint, while hidden layer width and accuracy tuning allow fine-grained trade-offs between performance and efficiency. Another important conclusion is that the size of a

KAN network can holistically be thought of in terms of its number of edges: this is proportional to the number of LUTs and FFs used, as seen in Figures 6(b) and 6(c). Together, these insights guide principled design choices for deploying KANELÉ under tight FPGA resource budgets.

5.7 Extension to Real-time Control Systems

To demonstrate that the KANELÉ paradigm extends well beyond the traditional supervised learning tasks typically studied in the LUT-based neural network community, we apply our framework to a reinforcement learning benchmark. Specifically, we evaluate on the HalfCheetah environment, a classic continuous control task

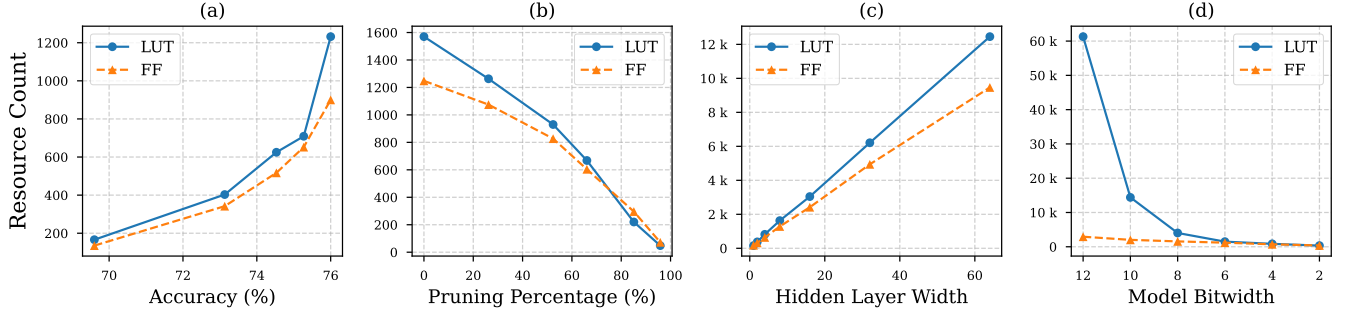


Figure 6: Ablation study of KANELE on the JSC OpenML benchmark demonstrating trade-offs between accuracy, pruning, and resource usage. (a) Accuracy improves steadily as hardware resources scale up with LUT and FF usage growing at roughly the same rate. (b) LUT/FF usage scales roughly linearly with the number of unpruned edges. (c) LUT/FF usage scales linearly with hidden layer width, confirming a direct mapping from learned activation functions (edges) to resources. (d) Decreasing activation bitwidths reduces LUT resource usage exponentially, with diminishing returns observed below 6 bits.

in reinforcement learning (RL), most commonly accessed through the MuJoCo physics simulator via OpenAI Gym (now Gymnasium) [38]. The objective is to learn a policy that enables a simulated two-legged agent to run as fast and as stably as possible. This environment is widely used as a standard benchmark to compare RL algorithms and function approximators. While HalfCheetah is a simulated benchmark, it captures key aspects of many practical robot control tasks, as they all rely on the same underlying design and physics principles.

This extension is motivated by prior work such as [23], where the authors showed that KAN actor networks with significantly fewer trainable parameters achieved higher rewards compared to much larger MLPs when trained with the Proximal Policy Optimization (PPO) algorithm. PPO is one of the most widely used RL algorithms in practice, particularly in robotics, games, and continuous control tasks.

5.7.1 Setup. Since the primary focus of this paper is inference efficiency, we incorporated KANs only as the function approximator for the policy (actor), as this is the component that must be deployed in practice. The value (critic) function remained an MLP for all experiments. We evaluated four training scenarios:

- (1) MLP actor + MLP critic (both FP),
- (2) Quantized MLP actor (8-bit) + MLP FP critic,
- (3) KAN FP actor + MLP FP critic,
- (4) Quantized KAN actor (8-bit) + MLP FP critic.

To ensure robustness against variance in random initialization, each scenario was trained with 5 different random seeds, for 1 million environment steps per seed. The actor networks were chosen such that the MLP actor had roughly five times more trainable parameters than the KAN actor (Table 6), highlighting that KAN can be more effective even with significantly fewer parameters.

5.7.2 Training results. Figure 7 shows the learning curves across the four scenarios. The quantized KAN actor achieves an average return of **2762.2**, outperforming both the larger MLP FP and quantized actor baselines (**1676.4** and **1558.8**) and the full-precision KAN actor (**2338.9**). These results demonstrate that KANs not only remain robust under aggressive 8-bit quantization, but can also

Table 6: Network architectures of the actor and critic.

Model	Dimensions	Trainable Parameters
MLP Actor	[17, 64, 64, 6]	5383
MLP Critic	[17, 64, 64, 6]	5383
KAN Actor	[17, 6]	1020

benefit from it, potentially due to regularization effects by fixed bit operations.

5.7.3 Hardware Performance. To evaluate the efficiency of KANs in practical deployment, we compare the hardware cost of the 8-bit quantized KAN actor against the 8-bit quantized MLP actor. The KAN actor was deployed using the KANELE framework, while the MLP actor was implemented with hls4ml [15] using the Resource strategy for a fair baseline. We planned to have both models synthesized on a Xilinx xczu7ev-ffvc1156-2-e FPGA in out-of-context mode. However, the 8-bit MLP design exceeds the available FPGA resources, so its performance results are based on HLS estimates rather than the actual implementation. Table 7 summarizes the results. As can be seen, the 8-bit KAN actor achieves significantly lower resource utilization, latency, and power compared to the 8-bit MLP actor, underscoring the advantages of KANs for real-time reinforcement learning control tasks.

In conclusion, the HalfCheetah task is a simulated environment that captures core principles of real-world control tasks and serves as a strong proxy for domains where real-time, resource-efficient policies are essential. These results therefore highlight the suitability of KANs for deployment in settings where such constraints matter (e.g., robotics, embedded control, trading, or quantum computing). Thus, the benchmark serves as a proof-of-concept: KANs can provide competitive or superior RL performance while being dramatically more efficient in terms of size and quantization tolerance.

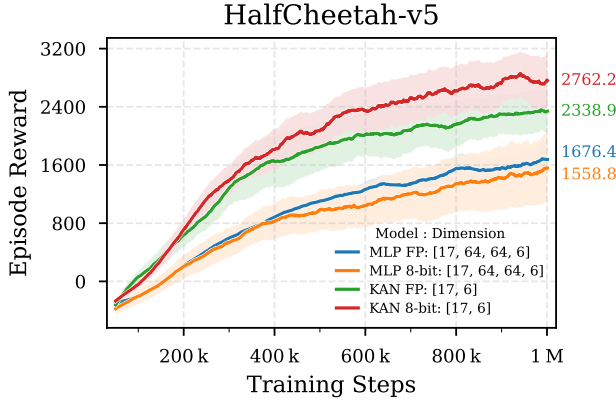


Figure 7: PPO training on HalfCheetah-v5 with 5 seeds. The quantized KAN actor (8-bit) outperforms both the KAN (FP) and the larger MLP (FP), despite using $\sim 5\times$ fewer parameters, showing robustness to quantization and strong parameter efficiency.

Table 7: FPGA performance of the KAN 8-bit model on the HalfCheetah RL task targeting the xczu7ev-ffvc1156-2-e FPGA. The MLP 8-bit actor does not fit on the FPGA, so its power estimate is unavailable and resource usage is reported from HLS estimates, while KAN 8-bit results are obtained after place-and-route in out-of-context mode.

Metric	KAN 8-bit	MLP 8-bit hls4ml
1M Episode Reward	2762.2	1558.8
Max. Frequency ($F_{\max}$)	884 MHz	500 MHz
Latency	4.5 ns	893 ns
BRAM	0	0
DSP	0	14,346
Flip-Flops (FF)	2,828	460,800
Look-Up Tables (LUT)	1,136	230,400
Area $\times$ Delay	1.3×10^4 LUT $\cdot$ ns	2.1×10^8 LUT $\cdot$ ns
Dynamic Power	0.224 nJ/sample	$\gg 0.224$ nJ/sample

6 Conclusion and Future Works

We present KANÉLÉ, a hardware–software co-design framework that maps Kolmogorov–Arnold Networks (KANs) onto a LUT-native computational architecture for FPGAs. Unlike most existing ML hardware–software co-design approaches, KANs are built entirely from learnable 1D activation functions defined on a fixed domain. Each learned activation function $\phi(x)$ is thus not merely approximated by a L-LUT: it is a lookup table. Moreover, the additive structure of KANs enables an especially natural form of pruning: each node can be directly removed from the summation without disrupting the remaining computation. This is in stark contrast to conventional LUT-based neural networks, where LUTs are typically chained together as indices into one another, making the removal of even a single LUT practically impossible without breaking the model. Consequently, mapping a KAN to an FPGA is less a process

of compilation and more one of direct instantiation. This paradigm shift—from emulating arithmetic to directly configuring logic—is what unlocks the extreme efficiency of KANÉLÉ, sidestepping the need for DSPs and BRAMs entirely and aligning the algorithm directly with the hardware’s native capabilities.

Across standard LUT–NN benchmarks and prior KAN–FPGA tasks, KANÉLÉ demonstrates strong performance with a favorable Area $\times$ Delay trade-off, offering substantially lower latency and reduced logic utilization compared to earlier KAN-on-FPGA designs, while matching or exceeding other LUT-centric architectures when the target function exhibits symbolic or physics-inspired structure. We further demonstrated the applicability of KANÉLÉ to real-time control by deploying an 8-bit KAN policy that surpasses a larger MLP while achieving ultra-low latency and higher resource efficiency on FPGA.

Future Works. Building on our results, we identify several avenues to broaden the scope and impact of KANÉLÉ:

- **Broader model families.** Extending beyond single KANs to ensembles, temporal and convolutional KANs, graph-based KANs, or transformer-style KANs (“KAN-GPT”) on FPGAs.
- **Alternative orthogonal bases.** Moving beyond B-splines by exploring Fourier, wavelet, or rational bases for learnable activations. These alternatives may improve approximation power and training dynamics while remaining LUT-compatible.
- **Practical deployment in control tasks.** Our reinforcement learning demo shows that 8-bit KAN policies can outperform larger MLPs while sustaining ultra-low FPGA latency, meeting demands of applications such as quantum error correction, plasma stabilization, adaptive optics, and robotics. Future work targets rapid in-field adaptation through hot-swapping edge tables via partial reconfiguration or LUT updates, enabling lightweight online learning with minimal latency.

Realizing the full potential of this paradigm, however, first requires overcoming the perception that KANs are inherently inefficient in hardware. Our work, KANÉLÉ, directly refutes this view by aligning the activation-centric KAN formulation with the native strengths of FPGA LUT fabrics. This approach transforms KANs into a practical, high-throughput, and power-efficient inference architecture. We believe this hardware-centric perspective not only solves a critical implementation challenge but also solidifies the promising path toward interpretable, low-resource neural networks that scale from embedded controllers to large scientific instruments, naturally benefiting from software–hardware co-design.

Acknowledgments

The authors thank the anonymous referees, Vladimir Loncar, Marta Andronic and Ryan Kastner for their valuable comments and helpful suggestions. PH, DH, and AG are supported by the NSF-funded A3D3 Institute (NSF-PHY-2117997).

References

- [1] 2020. Dry Bean. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C50S4B>.

- [2] Stefan Aeberhard and M. Forina. 1992. Wine. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5PC7J>.
- [3] Igor Aleksander, W. V. Thomas, and Pr Bowden. 1984. WISARD—a radical step forward in image recognition. *Sensor Review* 4 (1984), 120–124. <https://api.semanticscholar.org/CorpusID:108462259>
- [4] Marta Andronic and George A. Constantinides. 2023. PolyLUT: Learning Piecewise Polynomials for Ultra-Low Latency FPGA LUT-based Inference. In *2023 International Conference on Field Programmable Technology (ICFPT)*. IEEE. doi:10.1109/icfpt59805.2023.00012
- [5] Marta Andronic and George A. Constantinides. 2024. NeuralUT: Hiding Neural Network Density in Boolean Synthesizable Functions. In *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 140–148. doi:10.1109/fpl64840.2024.00028
- [6] Marta Andronic and George A. Constantinides. 2025. NeuralUT-Assemble: Hardware-aware Assembling of Sub-Neural Networks for Efficient LUT Inference. arXiv:2504.00592 [cs.LG] <https://arxiv.org/abs/2504.00592>
- [7] Alan T. L. Bacellar, Zachary Susskind, Mauricio Breternitz Jr., Eugene John, Lizy K. John, Priscila M. V. Lima, and Felipe M. G. França. 2025. Differentiable Weightless Neural Networks. arXiv:2410.11112 [cs.LG] <https://arxiv.org/abs/2410.11112>
- [8] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. 2021. MLPerf Tiny Benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks* (2021).
- [9] Alexander Dylan Bodner, Antonio Santiago Tepsich, Jack Natan Spolski, and Santiago Pourteau. 2025. Convolutional Kolmogorov–Arnold Networks. arXiv:2406.13155 [cs.CV] <https://arxiv.org/abs/2406.13155>
- [10] Hendrik Borrás, Giuseppe Di Guglielmo, Javier Duarte, Nicolò Ghielmetti, Ben Hawks, Scott Hauck, Shih-Chieh Hsu, Ryan Kastner, Jason Liang, Andres Meza, Jules Muhizi, Tai Nguyen, Rushil Roy, Nhan Tran, Yaman Umuroglu, Olivia Weng, Aidan Yokuda, and Michaela Blott. 2022. Open-source FPGA-ML codesign for the MLPerf Tiny Benchmark. arXiv:2206.11791 [cs.LG] <https://arxiv.org/abs/2206.11791>
- [11] CERN Collaboration. 2025. CERNBox LHC Jets Dataset. <https://cernbox.cern.ch/index.php/s/jvFd5MoWhGs15v/download> [Accessed: Sept 1, 2025].
- [12] Gonçalo G. Cruz, Balázs Renczes, Mark C. Runacres, and Jan Decuyper. 2025. State-Space Kolmogorov Arnold Networks for Interpretable Nonlinear System Identification. *IEEE Control Systems Letters* 9 (2025), 847–852. doi:10.1109/LCSYS.2025.3578019
- [13] Li Deng. 2012. The MNIST Database of Handwritten Digit Images for Machine Learning Research [Best of the Web]. *IEEE Signal Processing Magazine* 29, 6 (2012), 141–142. doi:10.1109/MSP.2012.2211477
- [14] Ivan Drokin. 2024. Kolmogorov–Arnold Convolutions: Design Principles and Empirical Studies. arXiv:2407.01092 [cs.CV] <https://arxiv.org/abs/2407.01092>
- [15] Farah Fahim, Benjamin Hawks, Christian Herwig, James Hirschauer, Sergio Jindariani, Nhan Tran, Luca P. Carloni, Giuseppe Di Guglielmo, Philip Harris, Jeffrey Krupa, Dylan Rankin, Manuel Blanco Valentin, Josiah Hester, Yingyi Luo, John Mamish, Seda Orgrenci-Memik, Thea Aarrestad, Hamza Javed, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, Sioni Summers, Javier Duarte, Scott Hauck, Shih-Chieh Hsu, Jennifer Ngadiuba, Mia Liu, Duc Hoang, Edward Kreinar, and Zhenbin Wu. 2021. hls4ml: An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices. arXiv:2103.05579 [cs.LG] <https://arxiv.org/abs/2103.05579>
- [16] Giuseppe Franco, Alessandro Pappalardo, and Nicholas J Fraser. 2025. *Xilinx/brevitas*. doi:10.5281/zenodo.3333552
- [17] Remi Genet and Hugo Inzirillo. 2025. TKAN: Temporal Kolmogorov–Arnold Networks. arXiv:2405.07344 [cs.LG] <https://arxiv.org/abs/2405.07344>
- [18] Xiao Han, Xinfeng Zhang, Yiling Wu, Zhenduo Zhang, and Zhe Wu. 2025. Are KANs Effective for Multivariate Time Series Forecasting? arXiv:2408.11306 [cs.LG] <https://arxiv.org/abs/2408.11306>
- [19] Yuntian Hou, Tianrui Ji, Di Zhang, and Angelos Stefanidis. 2025. Kolmogorov–Arnold Networks: A Critical Assessment of Claims, Performance, and Practical Viability. arXiv:2407.11075 [cs.LG] <https://arxiv.org/abs/2407.11075>
- [20] Abdullah Al Imran and Md Farhan Ishmam. 2024. FourierKAN outperforms MLP on Text Classification Head Fine-tuning. arXiv:2408.08803 [cs.CL] <https://arxiv.org/abs/2408.08803>
- [21] Tianrui Ji, Yuntian Hou, and Di Zhang. 2025. A Comprehensive Survey on Kolmogorov Arnold Networks (KAN). arXiv:2407.11075 [cs.LG] <https://arxiv.org/abs/2407.11075>
- [22] Alireza Khataei and Kia Bazargan. 2025. TreeLUT: An Efficient Alternative to Deep Neural Networks for Inference Acceleration Using Gradient Boosted Decision Trees. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*. ACM, 14–24. doi:10.1145/3706628.3708877
- [23] Victor Augusto Kich, Jair Augusto Bottega, Raul Steinmetz, Ricardo Bedin Grando, Ayano Yorozu, and Akihisa Ohya. 2024. Kolmogorov–Arnold Network for Online Reinforcement Learning. arXiv:2408.04841 [cs.LG] <https://arxiv.org/abs/2408.04841>
- [24] Chenxin Li, Xinyu Liu, Wuyang Li, Cheng Wang, Hengyu Liu, and Yixuan Yuan. 2024. U-KAN Makes Strong Backbone for Medical Image Segmentation and Generation. *arXiv preprint arXiv:2406.02918* (2024).
- [25] Longlong Li, Yipeng Zhang, Guanghui Wang, and Kelin Xia. 2025. Kolmogorov–Arnold graph neural networks for molecular property prediction. *Nature Machine Intelligence* 7, 8 (2025), 1346–1354. doi:10.1038/s42256-025-01087-7
- [26] Ziming Liu, Pingchuan Ma, Yixuan Wang, Wojciech Matusik, and Max Tegmark. 2024. KAN 2.0: Kolmogorov–Arnold Networks Meet Science. arXiv:2408.10205 [cs.LG] <https://arxiv.org/abs/2408.10205>
- [27] Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y. Hou, and Max Tegmark. 2025. KAN: Kolmogorov–Arnold Networks. arXiv:2404.19756 [cs.LG] <https://arxiv.org/abs/2404.19756>
- [28] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. arXiv:1711.05101 [cs.LG] <https://arxiv.org/abs/1711.05101>
- [29] Binglei Lou, Richard Rademacher, David Bolland, and Philip H. W. Leong. 2024. PolyLUT-Add: FPGA-based LUT Inference with Wide Inputs. arXiv:2406.04910 [cs.LG] <https://arxiv.org/abs/2406.04910>
- [30] Marmiton. n.d.. Cannelés bordelais. https://www.marmiton.org/recettes/recette_canneles-bordelais_11439.aspx. Accessed: Sept 23, 2025.
- [31] Igor D.S. Miranda, Aman Arora, Zachary Susskind, Luis A.Q. Villon, Rafael F. Katopodis, Diego L.C. Dutra, Leandro S. De Araújo, Priscila M.V. Lima, Felipe M.G. França, Lizy K. John, and Mauricio Breternitz. 2022. LogicWiSARD: Memoryless Synthesis of Weightless Neural Networks. In *2022 IEEE 33rd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 19–26. doi:10.1109/ASAP54787.2022.00014
- [32] Jennifer Ngadiuba, Vladimir Loncar, Maurizio Pierini, Sioni Summers, Giuseppe Di Guglielmo, Javier Duarte, Philip Harris, Dylan Rankin, Sergio Jindariani, Mia Liu, Kevin Pedro, Nhan Tran, Edward Kreinar, Sheila Sagar, Zhenbin Wu, and Duc Hoang. 2020. Compressing deep neural networks on FPGAs to binary and ternary precision with hls4ml. *Machine Learning: Science and Technology* 2, 1 (dec 2020), 015001. doi:10.1088/2632-2153/aba042
- [33] OpenML Contributors and LHC Jets HLF Curators. 2020. hls4ml lhc jets hlf (OpenML Dataset 42468). <https://www.openml.org/d/42468> [Accessed: Sept 1, 2025].
- [34] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 85 (2011), 2825–2830. <http://jmlr.org/papers/v12/pedregosa11a.html>
- [35] Shriyank Somvanshi, Syed Aaqib Javed, Md Monzurul Islam, Diwas Pandit, and Subasish Das. 2025. A Survey on Kolmogorov–Arnold Network. *Comput. Surveys* (June 2025). doi:10.1145/3743128
- [36] Zachary Susskind, Aman Arora, Igor D. S. Miranda, Luis A. Q. Villon, Rafael F. Katopodis, Leandro S. de Araújo, Diego L. C. Dutra, Priscila M. V. Lima, Felipe M. G. França, Mauricio Breternitz, and Lizy K. John. 2023. Weightless Neural Networks for Efficient Edge Inference. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques* (Chicago, Illinois) (PACT '22). Association for Computing Machinery, New York, NY, USA, 279–290. doi:10.1145/3559009.3569680
- [37] Zachary Susskind, Alan Bacellar, Aman Arora, Luis Villon, Renan Mendaña, Leandro Santiago, Diego Dutra, Priscila Lima, Felipe França, Igor Miranda, Mauricio Breternitz, and LIZY JOHN. 2022. Pruning Weightless Neural Networks. 37–42. doi:10.14428/esann/2022.ES2022-55
- [38] Emanuel Todorov, Tom Erez, and Yuval Tassa. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*. 5026–5033. doi:10.1109/IROS.2012.6386109
- [39] Van Duy Tran, Tran Xuan Hieu Le, Thi Diem Tran, Hoai Luan Pham, Vu Trung Duong Le, Tuan Hai Vu, Van Tinh Nguyen, and Yasuhiko Nakashima. 2024. Exploring the Limitations of Kolmogorov–Arnold Networks in Classification: Insights to Software Training and Hardware Implementation. In *2024 Twelfth International Symposium on Computing and Networking Workshops (CANDARW)*. 110–116. doi:10.1109/CANDARW64572.2024.00026
- [40] Yaman Umuroglu, Yash Akhauri, Nicholas J. Fraser, and Michaela Blott. 2020. LogicNets: Co-Designed Neural Networks and Circuits for Extreme-Throughput Applications. arXiv:2004.03021 [eess.SP] <https://arxiv.org/abs/2004.03021>
- [41] Yaman Umuroglu, Nicholas J. Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Visser. 2017. FINN: A Framework for Fast, Scalable Binarized Neural Network Inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA '17)*. ACM, 65–74. doi:10.1145/3020078.3021744
- [42] Erwei Wang, James J. Davis, Peter Y. K. Cheung, and George A. Constantinides. 2019. LUTNet: Rethinking Inference in FPGA Soft Logic. arXiv:1904.00938 [cs.LG] <https://arxiv.org/abs/1904.00938>
- [43] Yixuan Wang, Jonathan W. Siegel, Ziming Liu, and Thomas Y. Hou. 2025. On the expressiveness and spectral bias of KANs. arXiv:2410.01803 [cs.LG] <https://arxiv.org/abs/2410.01803>

- [44] Yizheng Wang, Jia Sun, Jinshuai Bai, Cosmin Anitescu, Mohammad Sadegh Eshaghi, Xiaoying Zhuang, Timon Rabczuk, and Yinghua Liu. 2025. Kolmogorov–Arnold-Informed neural network: A physics-informed deep learning framework for solving forward and inverse problems based on Kolmogorov–Arnold Networks. *Computer Methods in Applied Mechanics and Engineering* 433 (Jan. 2025), 117518. doi:10.1016/j.cma.2024.117518
- [45] Olivia Weng, Marta Andronic, Danial Zuberi, Jiaqing Chen, Caleb Geniesse, George A. Constantinides, Nhan Tran, Nicholas J. Fraser, Javier Mauricio Duarte, and Ryan Kastner. 2025. Greater than the Sum of its LUTs: Scaling Up LUT-based Neural Networks with AmigoLUT. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays* (Monterey, CA, USA) (FPGA '25). Association for Computing Machinery, New York, NY, USA, 25–35. doi:10.1145/3706628.3708874
- [46] Runpeng Yu, Weihao Yu, and Xinchao Wang. 2024. KAN or MLP: A Fairer Comparison. arXiv:2407.16674 [cs.LG] <https://arxiv.org/abs/2407.16674>
- [47] Jusheng Zhang, Yijia Fan, Kaitong Cai, and Keze Wang. 2025. Kolmogorov-Arnold Fourier Networks. arXiv:2502.06018 [cs.LG] <https://arxiv.org/abs/2502.06018>